_aula 9

Saindo do labirinto

Recapitulação, BFS e busca gulosa

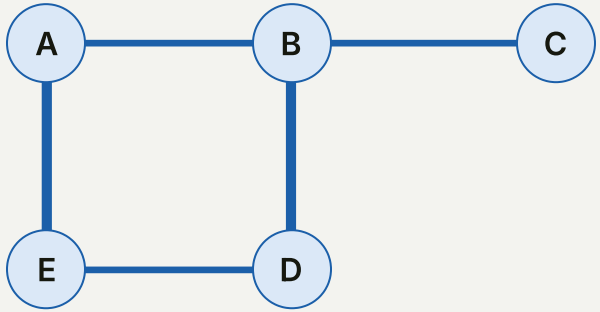
Prof. Pablo De Chiaro

algoritmos e estrutura de dados 2 (AEDII)

Prof. Pablo De Chiaro

O que é um grafo

Um conjunto de **vértices** ligados por **arestas**. Diferente da árvore, não tem raiz e pode ter ciclos.



lista de adjacência

```
grafo = {
    'A': ['B', 'E'],
    'B': ['A', 'C', 'D'],
    'C': ['B'],
    'D': ['B', 'E'],
    'E': ['A', 'D'],
}
```

Cada vértice guarda a lista dos seus vizinhos. Aresta não direcionada aparece nas duas listas.

O que é DFS busca em profundidade

```
def dfs(grafo, v, visitados):
    visitados.add(v)
    print(v)
    for w in grafo[v]:
        if w not in visitados:
            dfs(grafo, w, visitados)

dfs(grafo, 'A', set())
```

Avança o mais **fundo** possível. Sem vizinho novo, volta ao vértice anterior.

Estrutura: **pilha** (a própria recursão).

ordem de visita

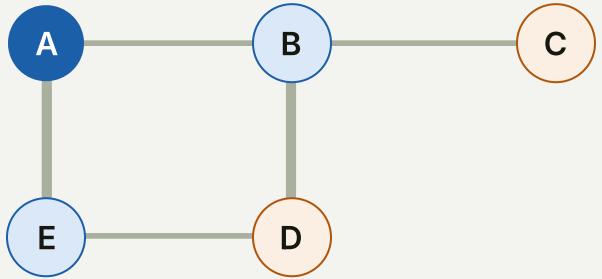
A B C D E

Prof. Pablo De Chiaro

BFS busca em largura · breadth-first search

Visita primeiro todos os vizinhos, depois os vizinhos dos vizinhos. Avança em **camadas** de distância a partir do início.

Estrutura: **fila**. Quem entrou primeiro sai primeiro.



camada 0: A camada 1: B, E camada 2: C, D

#	o que acontece	fila
1	começa em A	A
2	tira A, entram B e E	B E
3	tira B, entram C e D	E C D
4	tira E, D já está na fila	C D
5	tira C, depois D	vazia

ordem de visita

> **A B E C D**

BFS em Python

```
from collections import deque

def bfs(grafo, inicio):
    visitados = {inicio}
    fila = deque([inicio])
    while fila:
        v = fila.popleft()
        print(v)
        for w in grafo[v]:
            if w not in visitados:
                visitados.add(w)
                fila.append(w)
```

`popleft()` tira da frente, `append()` coloca no fim.

Marca como visitado ao **entrar** na fila, para nenhum vértice entrar duas vezes.

> Em grafo sem pesos, a BFS chega a cada vértice pelo **menor número de arestas**. A DFS não garante isso.

Prof. Pablo De Chiaro

Busca gulosa greedy best-first search

Conhece o destino e usa uma **heurística** h: uma estimativa de quanto falta. Sempre expande o candidato com o **menor h**.

```
import heapq

def h(v, saida):   # distância de Manhattan
    return abs(v[0] - saida[0]) + abs(v[1] - saida[1])

fronteira = [(h(inicio, saida), inicio)]
_, v = heapq.heappop(fronteira)   # menor h
```

Estrutura: **fila de prioridade**. Rápida, mas uma parede pode enganá-la: o caminho nem sempre é o menor.

h de cada casa até F

I h=6	5	4	3
5	4	3	2
4	3	2	1
3	2	1	**F**

I: início · F: fim. h = linhas + colunas que faltam até F, ignorando paredes.

O mesmo esqueleto, três escolhas

	DFS	BFS	Gulosa
estrutura	pilha	fila	fila de prioridade
próximo a sair	o último descoberto	o primeiro descoberto	o de menor h
conhece o destino?	não	não	**sim**
menor caminho?	não garante	**sim, sem pesos**	não garante

Prof. Pablo De Chiaro

Atividade a saída do labirinto

Cada casa livre do labirinto é um vértice. Duas casas livres vizinhas são ligadas por uma aresta. Encontrar a saída é buscar um caminho no grafo.

1

Criar o seu próprio labirinto, bem maior que o exemplo.

2

Implementar DFS, BFS e busca gulosa da entrada até a saída.

3

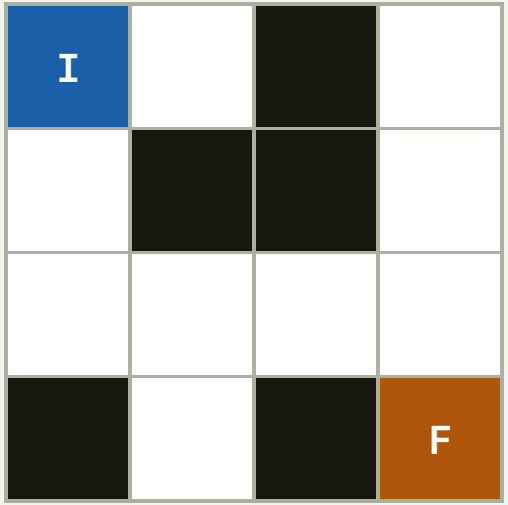
Comparar os caminhos encontrados e as casas visitadas.

Representando o labirinto

```
labirinto = [
    "I.#.",
    ".##.",
    "....",
    "#.#F",
]

def achar(lab, simbolo):
    for i, linha in enumerate(lab):
        if simbolo in linha:
            return (i, linha.index(simbolo))

inicio = achar(labirinto, 'I')   # (0, 0)
saida  = achar(labirinto, 'F')   # (3, 3)
```



`I`	início (entrada)
`F`	fim (saída)
`#`	parede
`.`	casa livre

Prof. Pablo De Chiaro

Do labirinto ao grafo

```
DIRECOES = [(-1, 0), (0, 1), (1, 0), (0, -1)]   # ↑ → ↓ ←

def livre(lab, i, j):
    dentro = 0 <= i < len(lab) and 0 <= j < len(lab[0])
    return dentro and lab[i][j] != '#'

def montar_grafo(lab):
    grafo = {}
    for i in range(len(lab)):
        for j in range(len(lab[0])):
            if not livre(lab, i, j):
                continue
            grafo[(i, j)] = []
            for di, dj in DIRECOES:
                if livre(lab, i + di, j + dj):
                    grafo[(i, j)].append((i + di, j + dj))
    return grafo

grafo = montar_grafo(labirinto)
```

saída

```
>>> grafo[(0, 0)]
[(0, 1), (1, 0)]
>>> grafo[(2, 2)]
[(2, 3), (2, 1)]
```

Mesmo formato da aula 8: dicionário de listas. Agora os vértices são tuplas `(linha, coluna)`.

Prof. Pablo De Chiaro

Desenhando um caminho

```
def desenhar(lab, caminho):
    grade = [list(linha) for linha in lab]
    for i, j in caminho:
        if grade[i][j] == '.':
            grade[i][j] = '*'
    for linha in grade:
        print(''.join(linha))

# caminho escrito à mão, só para testar
caminho = [(0, 0), (1, 0), (2, 0), (2, 1),
           (2, 2), (2, 3), (3, 3)]
desenhar(labirinto, caminho)
```

saída

```
I.#.
*##.
****
#.#F
```

Na atividade, o `caminho` é o que cada algoritmo devolve.

Prof. Pablo De Chiaro

O seu labirinto

1. Pelo menos **10 × 10**, na mesma notação: `I F #  .`
2. Pelo menos **duas rotas** até a saída, de tamanhos diferentes.
3. Alguns **becos sem saída**, para a busca precisar voltar.
4. Reaproveitar `montar_grafo` e `desenhar` sem alterar.

> **dica**
>
> A DFS segue a ordem de `DIRECOES`: tenta cima e direita primeiro. Coloque a rota longa desse lado e compare o que cada busca encontra.

Prof. Pablo De Chiaro

O que implementar

```
labirinto = [...]   # o seu, mínimo 10 × 10
grafo = montar_grafo(labirinto)

def dfs(grafo, inicio, saida): ...
def bfs(grafo, inicio, saida): ...
def gulosa(grafo, inicio, saida): ...
# cada uma devolve (caminho, casas_visitadas)

desenhar(labirinto, caminho)   # para cada uma
```

Criar o próprio labirinto, desenhar os três caminhos e montar uma tabela com o tamanho de cada caminho e o número de casas visitadas.

na correção oral, explicar

a por que a BFS encontrou o menor caminho e a DFS não;

b em que ponto do seu labirinto a gulosa foi enganada por uma parede;

c como o caminho é reconstruído a partir das casas visitadas.